

Advanced signal processing

Exam 2

Name	Initials	Student number	Retning
Alexander Nygaard Thomsen	AT	201911329	E

Due date: 12/11/2026

1 Introduction

This lab journal will take a shallow dive in to adaptive signal processing, specifically comparison of three common types of implementation, the regular *LMS*(REF), the *leaky-LMS*(REF) and the *Variable step-size LMS*(REF). Comparison of their performances in an experimental sense, a delve into their specifics, use cases and a critical discussion.
It is a assumed that the reader has some basic understand of digital filtering and a general grasp of their implementation, therefore a minimum amount of effort will be spend on that topic, however characterics regarding these mentioned types of filters, will be explained and discussed.
Filtering in general, is the art of removing the unwanted or highlighting and improving specific feature of input signals. Mainly in the analog domain, a filter would be designed for given frequency or bandwidth, with little possibility of expanding or changing. Such is the same problem when designing FIR(REF) or IIR(REF) filters in the digital domains. The lack of ability to adapt the characterics of the filtering gave birth to the aptly named *Adaptive filter*.

2 Theoretical background

Contrary to regular filtering, the designer targets a specific frequency or bandwidth, placing zeroes and poles on the unit-circle to *approximate*, a desired result, or atleast get somewhat close to something acceptable.

2.1 LMS

In adaptive filtering, we're more concerned about the system as an input and how we implement an algorithm to force the error to be zero. In this lab journal the overall system we'll be examining will take the form of:

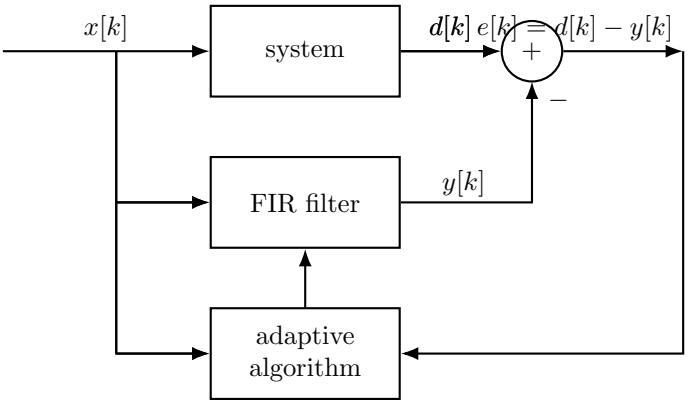


Figure 1: Adaptive FIR filter structure.

Where $x[k]$ will be some randomly generated noise, with a zero-mean, unit variance, passing through a system summed together with a sinusoid $s[k]$, for testing purposes, to experimentally see how the system responds to both noise and a signal combined.

In this example, the unidentified system will be $d[k] = 0.5x[k] + 0.25x[k-1] - 0.3x[k-2]$

The simple LMS filter works by taking the output of the system, as the desired output $d[k]$ and then subtracting $y[k]$, which is the output of the FIR filter, governed by a vector of weights $w[k]$. The The LMS algorithm will then try to force the error $e[k] = d[k] - y[k] \approx 0$ by tuning the weights in descrete steps.
The vector $\mathbf{W}[k]$ hold the coefficients and the vector $\mathbf{X}[k]$ holds old samples :

$$\mathbf{W}[k] = \begin{bmatrix} w_0[k] \\ w_1[k] \\ ... \\ w_N - 1[k] \end{bmatrix}, \qquad \mathbf{X}[k] = \begin{bmatrix} x[k] \\ x[k-1] \\ ... \\ x[k-N+1] \end{bmatrix}$$

The complete algorithm for computing the coefficiens for the vector:

$$\mathbf{W}[k+1] = \mathbf{W}[k] + 2\mu e[k]\mathbf{x}[k]$$

The factor μ is factor that gives some control of how aggressize the filter should act, as it a scalar that is multiplied onto the weights and the old samples. The idea of the algorithm is then, that each iteration that it runs, the weights of the filter are multipled with the signal, which affects how $y[k+1]$ samples will behave, this affects the error, and it repeats. Each iteration the weights gets updates aswell, so that they are applied with a different value at the next iteration.

2.2 Variable Step-size LMS (VVS LMS)

As briefly mentioned in the LMS section, the factor μ is a deciding factor in filter performance for an LMS filter. It determines the speed of convergence and accuracy. In the variable step-size algorithm each filter tab has an independent. It follows the form:

$$w_i[n+1] = w_i[n] + 2\mu_i[n]e[n]x[n-i], \qquad for \qquad i = 0, 1, \ldots, N-1 \tag{1}$$

The interesting part of this implementation is the gradient term, which is an intermediate calculation for the update of the μ value.

$$g_i[n] = e[n]x[n-i] \tag{2}$$

and

$$u_i[n] = u_i[n-1] + \rho sign\left[g_i[i]\right] sign\left[lg_i[n-1]\right] \tag{3}$$

It is then up to the individual use case to determine a μ_{min} and μ_{max} This essentailly calculates a gradient for direction, positive or negative, that the μ should move, granting the ability to increase or decrease the value. Another tuneable factor is ρ which essentially dictates how much the step-size itself changes, as it sits as an index independt factor, just as μ did in the regular LMS.

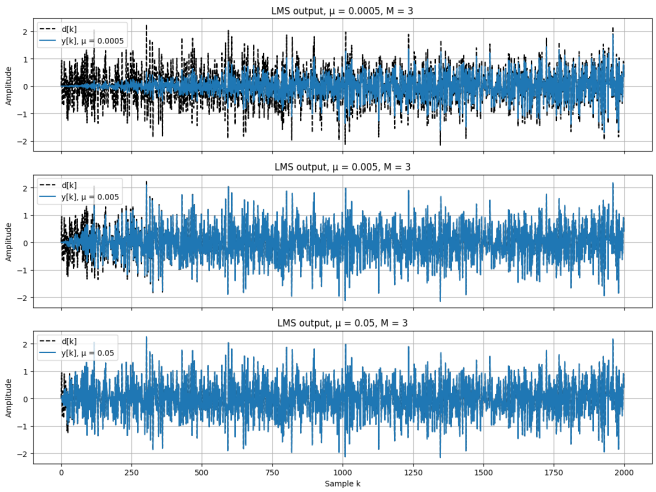
3 Numerical experiment

3.1 LMS experimentation

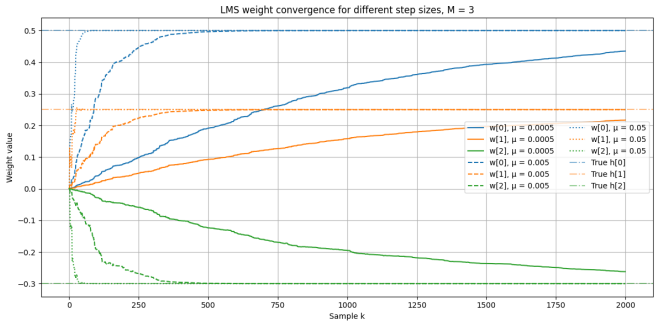
Looking at the equations for the LMS filter, we're given some handles to tune the filter with namely μ, M the amount of filter taps and N how long the algorithm will run.

For the initial tests, we'll keep the amount of filter tabs fixed to $M = 3$, and we'll instead vary the step size by a factor of 1000, and keep the test running for a 2000 samples. This is an estimate to have enought samples to atleast at higher values for μ that the filter will converge.

The next experiment is increasing the filter taps, where $M = [3, 5, 10]$. This is done to observe if increasing the filter tabs, beyond the amount of poles present in the unidentified system, has any influence on either convergence speed and accuracy.

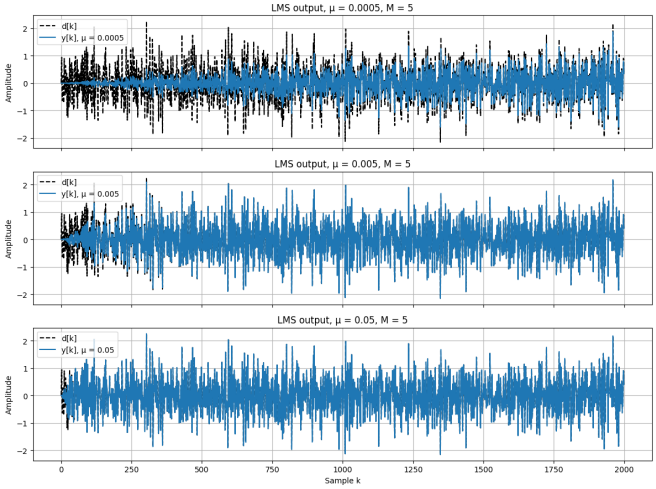


(a) LMS, M=3

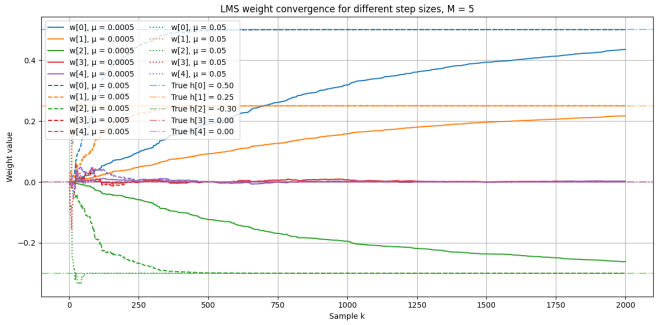


(b) LMS, weights, M=3

Figure 2: Comparison of sizes of μ for M=3

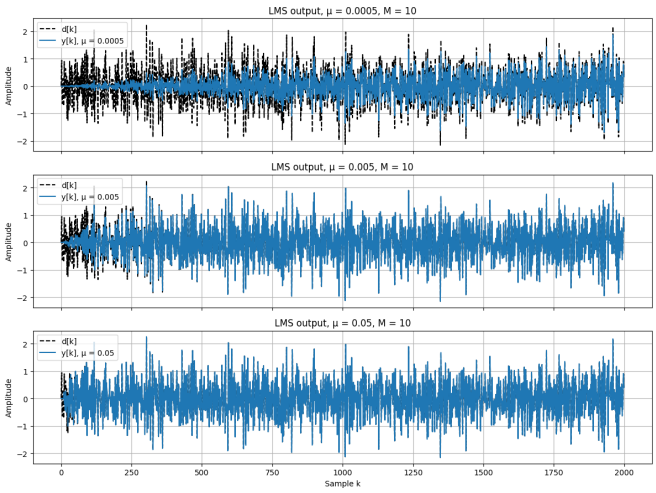


(a) LMS, M=5

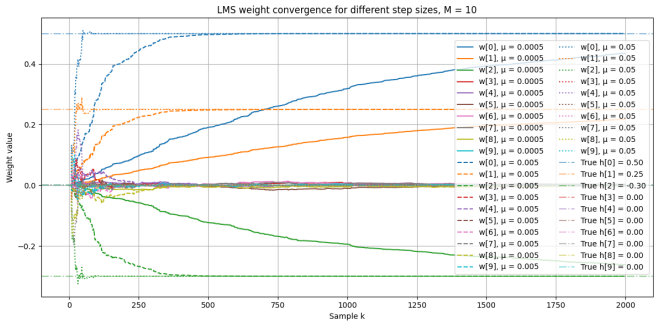


(b) LMS, weights, M=5

Figure 3: Comparison of sizes of μ for M=5



(a) LMS, M=10



(b) LMS, weights, M=10

Figure 4: Comparison of sizes of μ for M=10

3.2 variable step-size experimentation

The experiment with the variable step-size implementation, will be conducted in a slightly similar manner, however, in the regular LMS, we are *almost* free to choose our values of μ, however in the case for the variable step-size, some bounds are defined, as qouted [INDSÆT REFERENCE TIL BOGEN] *"The minimum bound, which may be imposed on the variable step-size parameters $\mu_i(n)'s$, can be as low as zero. However in actual practice, a positive bound is usually used"*. Therefore stability is ensured if $trace(\mu[n]\mathbf{R}) < \frac{1}{3}$. Which is the linear sum of the diagonal of the autocorrelation matrix and μ vector:

$$\mu_0[k]R_{00} + \mu_1[k]R_{11} + \cdots + \mu_{M-1}[k]R_{M-1,M-1} < \frac{1}{3}$$

For white noise the autocorrelation matrix is:

$$\mathbf{R} = \sigma_x^2\mathbf{I}$$

Because the unidentified system actually is 3 terms, $M = 3$ This creates the constrains:

$$\mu_i \leq \mu_{max}$$

Where the largest sum of values becomes:

$$\sum_{i=0}^{M-1} \leq M\mu_{max}$$

This can rearranged:

$$\mu_{max} < \frac{1}{3M\sigma_x^2}$$

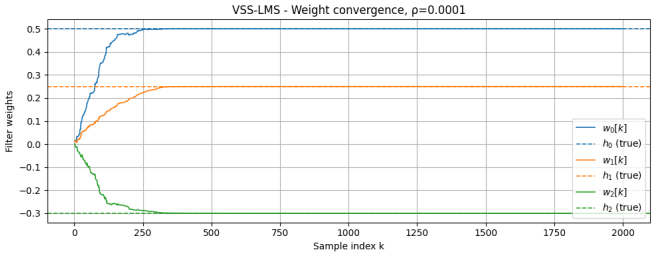
Since we already established the variance for the noise:

$$\mu_{max} < \frac{1}{3M}$$

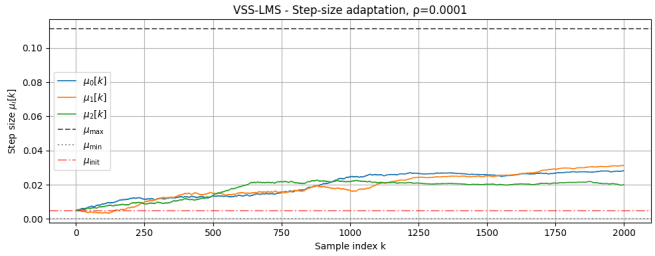
And as already mentioned, in this case $M = 3$, leads To

$$\mu_{max} = \frac{1}{3 \cdot 3} = \frac{1}{9} = 0.1111$$

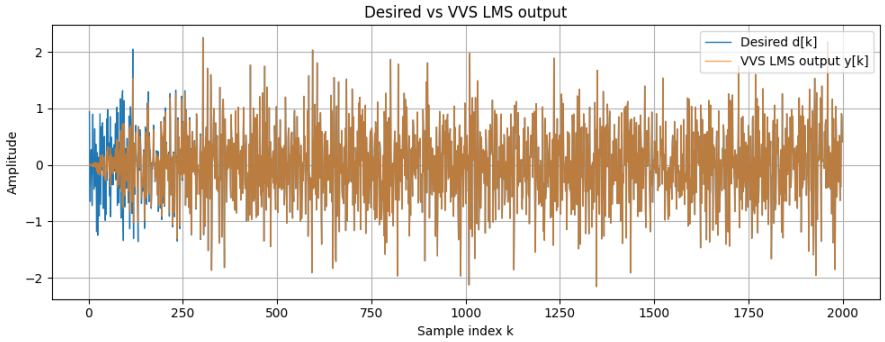
Instead changes to ρ can be applied, thus our test cases will be varying values of this factor in small increments of 0.0001, 0.001 and 0.005.



(a) Filter weight convergence, $\rho = 0.0001$

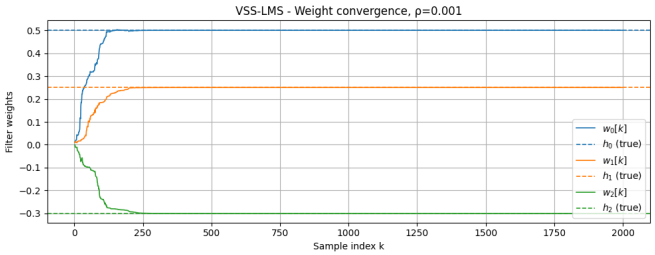


(b) filter step size adaptation, $\rho = 0.0001$

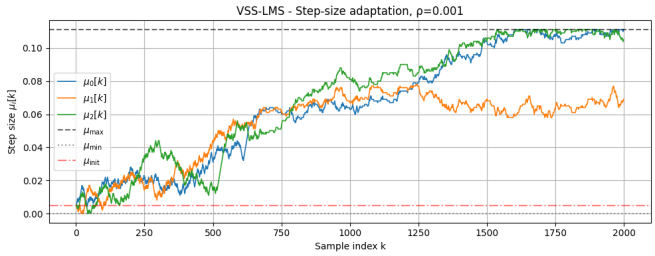


(c) VVSLMS output, $\rho = 0.0001$

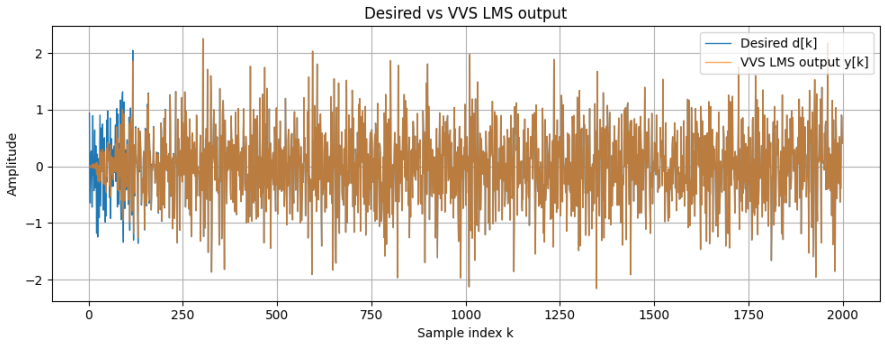
Figure 5: Results for running VVS LMS with 2000 samples



(a) Filter weight convergence, $\rho = 0.001$

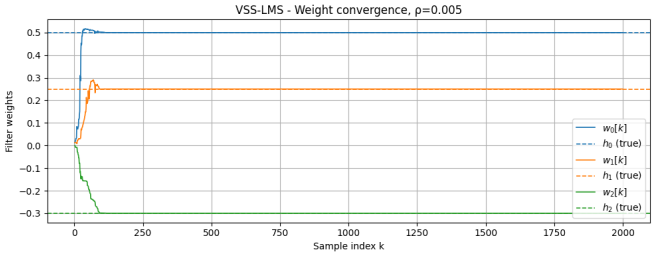


(b) filter step size adaptation, $\rho = 0.001$

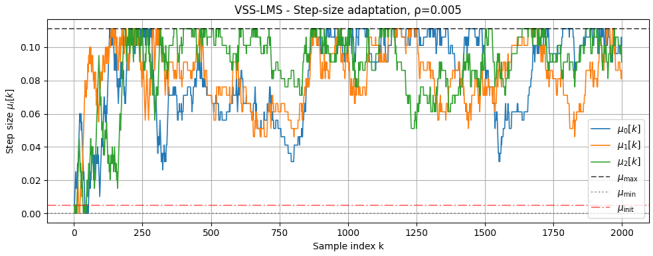


(c) VVSLMS output, $\rho = 0.001$

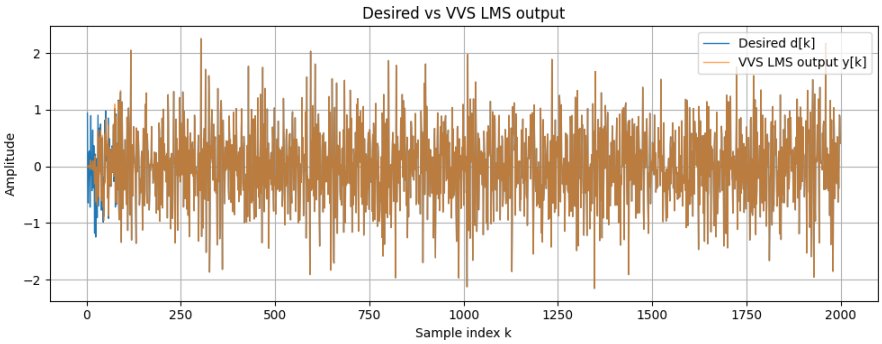
Figure 6: Results for running VVS LMS with 2000 samples



(a) Filter weight convergence, $\rho = 0.005$



(b) filter step size adaptation, $\rho = 0.005$



(c) VVSLMS output, $\rho = 0.005$

Figure 7: Results for running VVS LMS with 2000 samples

4 Results

4.1 Analysis

4.1.1 LMS

Starting with the output of the regular LMS filter, it can be seen on figure:2, where the size of the μ is very small and how it compares to an increase in size in an order og magitude twice. Towards the end of the 2000 samples, $y[k]$ seems to almost be equal to that of $d[k]$ meaning that the error, which is the difference of two would become zero, this would essentailly mean that the filter that converged on the weights of the unidentified system. However as the value of μ increases, the speed at which this happens increases, quite dramatically.

Taking a look at the graph for the weights, the semi dashed lines, belonging to $\mu = 0.005$, the weights have almost converged after 500 samples, which is a fourth of the total amount of samples. And again, an increases in this speed, or decrease in time, can be see if the value is even larger. Looking at the equation for the update of the weights, μ is essentailly just scaling how much we want the product of the error and the old samples, to affect the future weights. As this goes on for every iteration, it becomes intuitive how it slows the entire process down.

We can consider an example where we let every input sample be $x[k] = 1$ and we have only the first term in our unidentified system with the difference equation $d[k] = 0.5x[k]$, and the filter is initialized to be $w[0] = 0$. We want to find the $w_{ideal} = 0.5$ such that the error is forced to 0. A entire iteration with $\mu = 0.05$ of the LMS becomes:

$$d[k] = 0.5(1) = 0.5$$

$$y[0] = w[k]x[k] = 0(1) = 0$$

$$e[k] = 0.5 - 0 = 0.5$$

$$w[k+1] = w[k] + 2\mu e(k)x(k) = 0 + 2(0.05)(0.5)(1) = 0.05$$

Table 1: comparison of LMS weight update

k	$\mu = 0.05$	$\mu = 0.4$
0	0.0000	0.0000
1	0.0500	0.4000
2	0.0950	0.4800
3	0.1355	0.4960
4	0.1720	0.4992
5	0.2048	0.4998
6	0.2341	0.4999
7	0.2609	0.4999
8	0.2848	0.4999
9	0.3063	0.5000
10	0.3257	0.5000

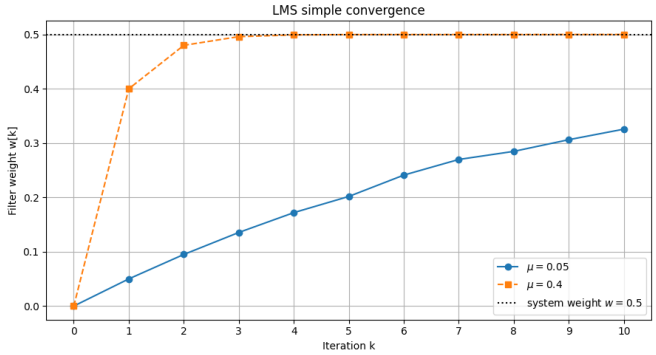


Figure 8: Weight convergence of

What table 1 shows is that with the large step size of $\mu = 0.4$, we're already 80% of the way towards the convergence after the first iteration, but getting closer to the ideal weight become harder. In this particular case, the convergence is nicely approaching the ideal coefficients, this is due to the constant input of 1. However if the system gets a sudden impulses added to the input, then it is easy to see that the larger value of μ wont *dampen* the larger step input, but larger values of μ also comes with some drawbacks that will be expanded upon in the discussion section.

4.2 VVS LMS

Looking at the results from the experiment on figure: 5, 6 and 7. Its difficult to see any difference between the variable step-size and the regular implementation of the LMS. However, one is to look at the graph for the weight convergence, this might a clearly indication of how they differ in this scenario. The most direction comparison is between figure 2b, and any of the weight graphs 5a, 6a and 7a.
These graphs tracks the changes in the filter coefficients and shows how the filter adapts to the unknown system. The difference here is that all implementations of the variable step-size version seems steeper, this gives an indication that the filter reacts faster, however the evolution of the weights seems more sporadic than the regular LMS implementation. This phenomenon will be expanded upon the discussion section.

Looking at the evolution of the step-size adaptation for each of the experiments on figure: 5b, 6b and 7b. This shows how the value of μ increases over time, as mentioned earlier, there is a minimum and maximum bound for μ in the VVS-LMS, however only with values of $\rho = [0.001, 0.005]$ does it actually approach the upper limit, which is then being enforced by the algorithm itself.

4.3 Discussion

As alluded to in the LMS analysis section, larger values of μ is not necessary a good thing. If too a large is selected, the resulting product of the input and the error is essentailly gained by the value, if this value $\mu \leq 1$ it is easy to see that, this will cause the resulting calculations to start spiralling out of control. An optimum value for μ can be derived, however as long as:

$$0 < \mu < \frac{1}{\lambda_{max}}$$

This is equation (6.17) from the adaptive filter book, [INDSÆT REFERENCE TIL BOGEN] "where the λ_{max} is the maximum eigenvalue of $\mathbf{R}$", where $\mathbf{R}$ is the autocorrelation matrix.
To observe how well the regular LMS performs in our experiments, we've taken a look at much $y[k]$ approches $d[k]$ which would result in the error converging to zero. This can be represented as the mean square error instead, and when this is done over multiple runs of the filter. (INDSÆT REFERENCE TO WIKI: MEAN_SQUARED_ERROR)

$$MSE = \frac{1}{n}\sum_{i=1}^{n} e_i^2[k]$$

Letting this complete a 1000 runs, we produce a new expression for the estimate of the expected squared error at each sample interval. This results in the following graph:

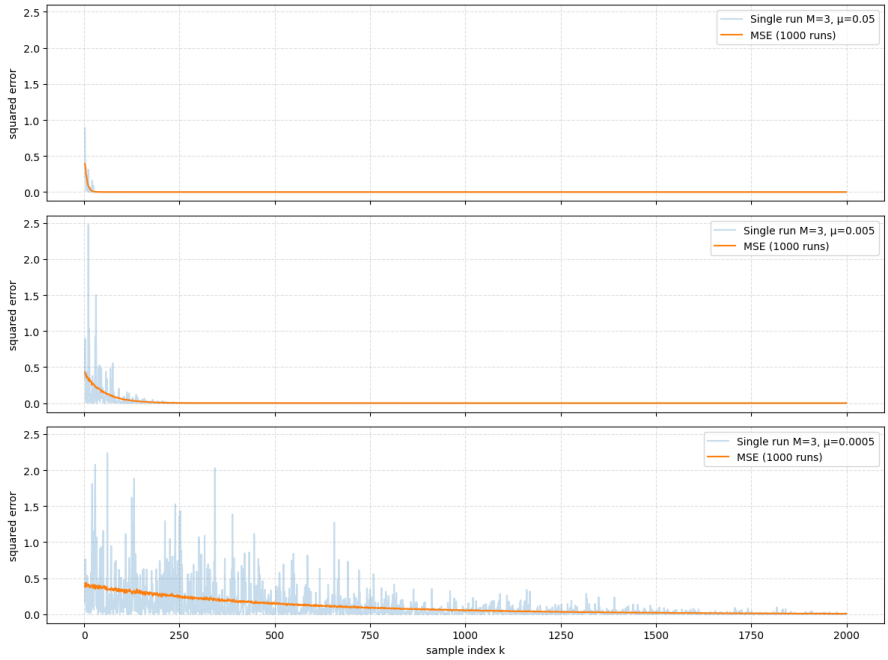


Figure 9: The mean squared error of the LMS, with varying sizes of μ

On figure 9 it is possible to see the decreasing nature of the error, it seems smooth, however this averages over 1000 runs, this does not mean that the filter itself will be smooth, but over the averaged runs, it is. The reason for this type of result is also the choice of input, where white gaussian noise is exhibits some fluctuations, when averaged over that many runs, it will average out smoothly. The curve also shows the filters ability, in this case to converge towards 0, meaning the filter has approximated the unidentified system.

An interesting observation can made when comparing figure:5b and 5a. It can be seen that the weights of the filter have converged relatively fast, however the step-size adaptation continues to grow, long after the weights. This is a phenomenon unique to the VSS-LMS. Taking a look at the equations for updating the weights $w_i[n+1]$: 1 and the μ update equation 3 from earlier. This particular implementation contains the function $sign(g_i[k])$ and $sign(g_i[k-1])$ this is used to estimate the gradient of the error, resulting in

$$\Delta\mu_i[k] = \begin{cases} +1 & g_i[k] \text{ and } g_i[k-1] \qquad \textit{same sign} \\ -1 & g_i[k] \text{ and } g_i[k-1] \qquad \textit{opposite sign} \\ 0 & \text{Either gradient is zero} \end{cases} \tag{4}$$

The previous sample from $g_i[k]$, then could be a very small number, if an assumption is made that the filter weights have converged. Then only very small changes will make it into the $g_i[k]$ equation, but since the μ update only cares about the sign, this will either add or subtract ρ from μ, however in the weight update equation, ρ is just a factor with $e[n]x[n-1]$, which are very small numbers , thus the weight update will become even smaller. And this process repeats for every iteration. This leads to a situation where a adapted filter, will produce very small errors, but can accidentally start to integrate a large value of μ, much like a wind-up mechanism in control systems. However this is mainly due to the fact that the input of the algorithm is being fed with gaussian noise.

5 Conclusion